

PTW LAB I	Kucharczyk Wojciech Album 57780 Grupa 250A	Wersja 2026-05-06
---------------------------------	---	------------------------------

SERVER-SIDE RENDERING Z WYKORZYSTANIEM NODE.JS

SPIS TREŚCI

Spis treści.....	1
Cel zajęć.....	1
Rozpoczęcie.....	1
Uwaga.....	2
Ponowne klonowanie repozytorium.....	2
Pobranie i uruchomienie PNPM.....	2
Pierwsze kroki z Express.js.....	2
Routingi i szablony.....	4
Portowanie aplikacji z LAB G.....	7
Implementacja.....	9
Commit projektu do GIT.....	12
Podsumowanie.....	13

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- wykorzystanie menedżera pakietów pnpm;
- instalacja i uruchomienie Express.js;
- zarządzanie routingami w Express.js;
- wykorzystanie szablonów EJS;
- portowanie aplikacji webowych CRUD pomiędzy PHP a JS.

W praktycznym wymiarze uczestnicy wykorzystają prosty framework Express.js do odtworzenia funkcjonalności naiwnego frameworka z poprzedniego laboratorium.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie zasad instalacji i uruchamiania programów w node. Powtórzenie zasad działania npm i pnpm. Różnice pomiędzy npm i pnpm.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do **Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe** i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub **Ctrl+A -> F9**.

PONOWNE KLONOWANIE REPOZYTORIUM

W związku z problemami wydajnościowymi w odczycie plików na macierzy dyskowej **I:**, to laboratorium również musimy wykonać na pulpicie. Jest to **bardzo zła** praktyka, ale niestety administratorom nie udało się znaleźć innego rozwiązania ani permanentnego obejścia. Wyłącznie jeśli korzystasz z pulpitu zdalnego do realizacji zadań, upewnij się, że Twoje repozytorium jest sklonowane na pulpit, tj. do katalogu analogicznego do **C:\Users\akarczmarczyk\Desktop\ptw**.

Na tym etapie warto w ustawieniach tej karty pracy zmienić wartość niestandardowej właściwości **labsdir** oraz **labdir**, po czym odświeżyć wszystkie pola w dokumencie **Ctrl+A, F9**.

POBRANIE I URUCHOMIENIE PNPM

W tym zadaniu przećwiczymy korzystanie z innego menedżera pakietów dla node – **pnpm**. Otwórz terminal i przejdź do katalogu repozytorium, tj. **C:\Users\akarczmarczyk\Desktop\ptw**. Zainstaluj **pnpm** poleceniem:

```
npm install -g pnpm@latest-10
```

Następnie sprawdź poprawność instalacji poleceniem:

```
pnpm -v
```

Powinna wyświetlić się wersja programu, przykładowo:

```
PS C:\Users\akarczmarczyk\Desktop\ptw> pnpm -v
10.33.4
```

PIERWSZE KROKI Z EXPRESS.JS

Utwórz katalog **C:\Users\akarczmarczyk\Desktop\ptw\lab-i\hello**. Przejdź do niego terminalem i zainstaluj **Express.js** w wersji 5:

```
pnpm install express@5
```

Spodziewany efekt:

```
PS C:\Users\akarczmarczyk\Desktop\ptw\lab-i\hello> pnpm install express@5
Packages: +65
Progress: resolved 65, reused 0, downloaded 65, added 65, done

dependencies:
+ express 5.2.1

Done in 2.3s using pnpm v10.33.4
```

Następnie doinstaluj zależność **prettier**:

```
pnpm install prettier@3
```

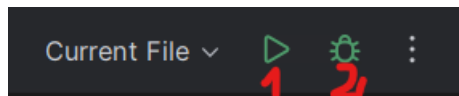
Następnie otwórz katalog `C:\Users\akarczmarczyk\Desktop\ptw\lab-i\hello` w PhpStorm. Utwórz plik `hello.js` o następującej treści (koniecznie wykorzystaj swój numer albumu jako numer portu):

```
import express from 'express'
const app = express()
const port = 57780
app.get('/', (req, res) => {
  res.send('Hello World from Express!')
})
app.listen(port, () => {
  console.log(`Hello World app listening on port ${port}`)
})
```

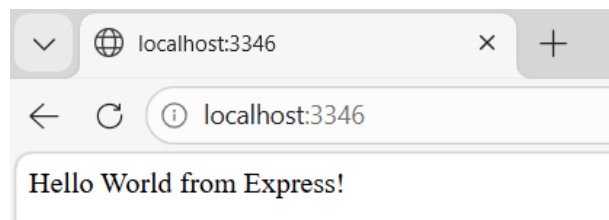
Teraz skonfigurujemy Prettier. Wejdź do ustawień PhpStorm i w Languages & Frameworks > JavaScript > Prettier wybierz konfigurację automatyczną. Zapisz zmiany. Kliknij w kod `hello.js` i wykorzystaj skrót klawiszowy `Ctrl + Alt + Shift + P`. Kod powinien ulec automatycznemu reformatowaniu.

Cofnij i ponów operację kilka razy. Jakie zmiany zachodzą po wykorzystaniu Prettiera?

Teraz uruchom serwer klikając na przycisk (1) poniżej:

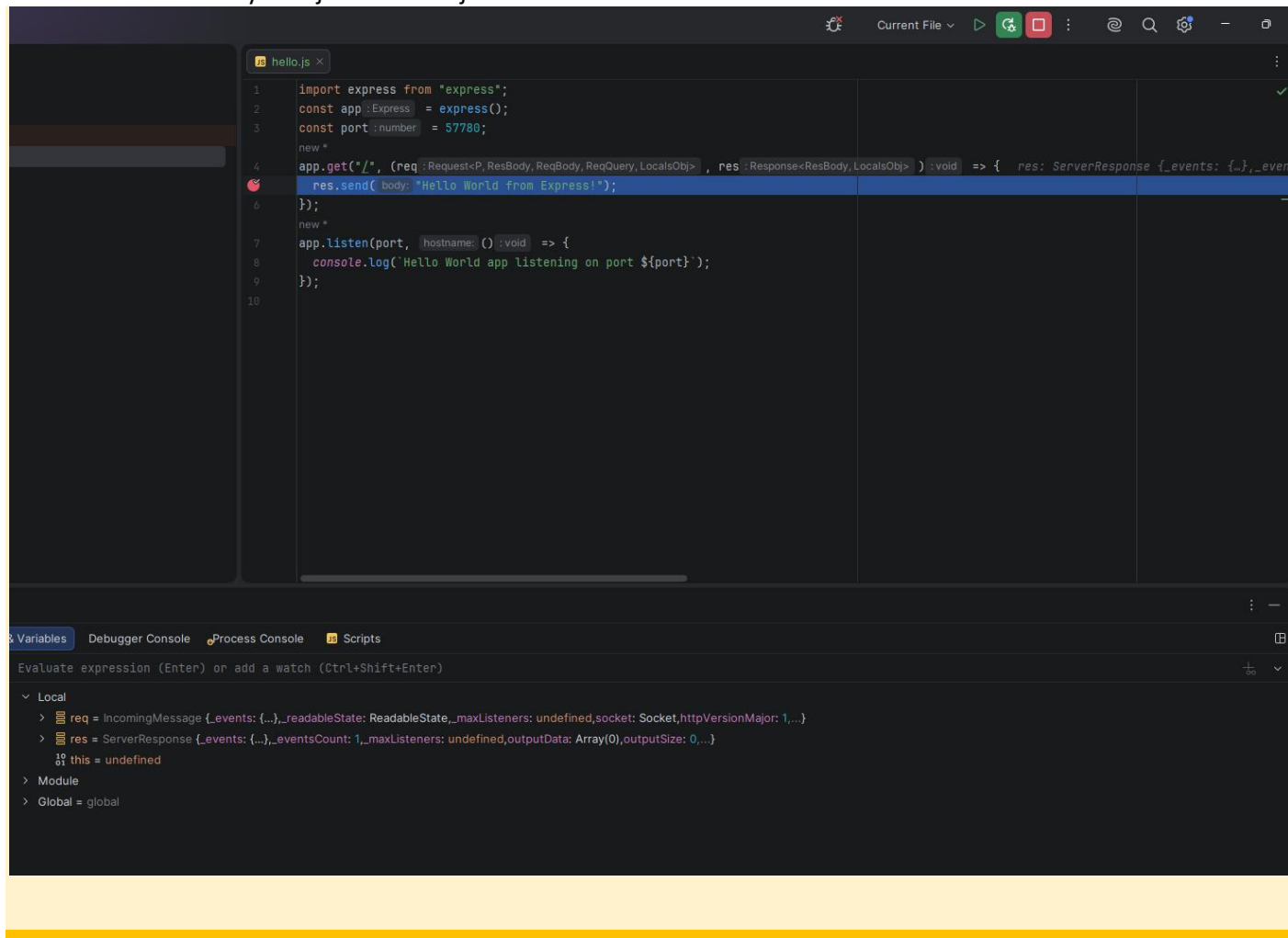


Przeczytaj w panelu logów, czy uruchomienie serwera powiodło się, a następnie otwórz w przeglądarce stronę `http://localhost:57780`. Oczekiwany efekt:



Przetestujemy teraz debugowanie. Ustaw pułapkę w handlerze route'a `/` (linia 5). Zatrzymaj serwer i uruchom ponownie przyciskiem (2) powyżej. Ponownie wejdź na stronę `http://localhost:57780`. Aplikacja powinna zatrzymać się, a panel **Threads & Variables** wypełniony będzie dostępnymi zmiennymi i ich wartościami.

Wstaw zrzut ekranu kodu aplikacji w PhpStorm, z uruchomionym debuggerem, zatrzymanym wewnątrz handlera ścieżki `„/”`. Upewnij się, że kod jest sformatowany Prettierem, debugger faktycznie jest zatrzymany we wskazanej linii, a w panelu **Threads & Variables** dostępne są bieżące wartości zmiennych.



Punkty:

0

1

ROUTINGI I SZABLONY

Obecnie aplikacja obsługuje tylko jedną ścieżkę (/), a zwracana odpowiedź jest zakodowana w handlerze na sztywno. W kolejnym kroku dodamy nowy routing oraz wykorzystamy szablony EJS.

Zainstaluj w projekcie EJS:

```
pnpm install ejs@5
```

Zmodyfikuj kod aplikacji zgodnie z poniższym:

<pre> import express from "express"; const app = express(); const port = 3346; app.get("/", (req, res) => { res.send("Hello World from Express!"); }); app.listen(port, () => { console.log(`Hello World app listening }); </pre>	<pre> 1 2 3 >> 4 5 6 >> 7 8 9 10 </pre>	<pre> import express from "express"; const app = express(); const port = 3346; app.set("view engine", "ejs"); app.get("/", (req, res) => { res.send("Hello World from Express!"); }); app.get("/hello/:name", (req, res) => { const name = req.params.name; res.render("hello", { name }); }); app.listen(port, () => { console.log(`Hello World app listening o }); </pre>
---	---	--

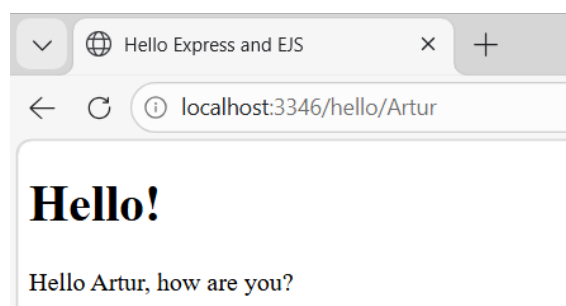
Następnie utwórz plik `views/hello.ejs` o następującej treści:

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Hello Express and EJS</title>
</head>
<body>
<h1>Hello!</h1>
<p>Hello <%= name %>, how are you?</p>
</body>
</html>

```

Zrestartuj serwer i wejdź na stronę `http://localhost:57780/hello/Wojciech`. Spodziewany efekt:



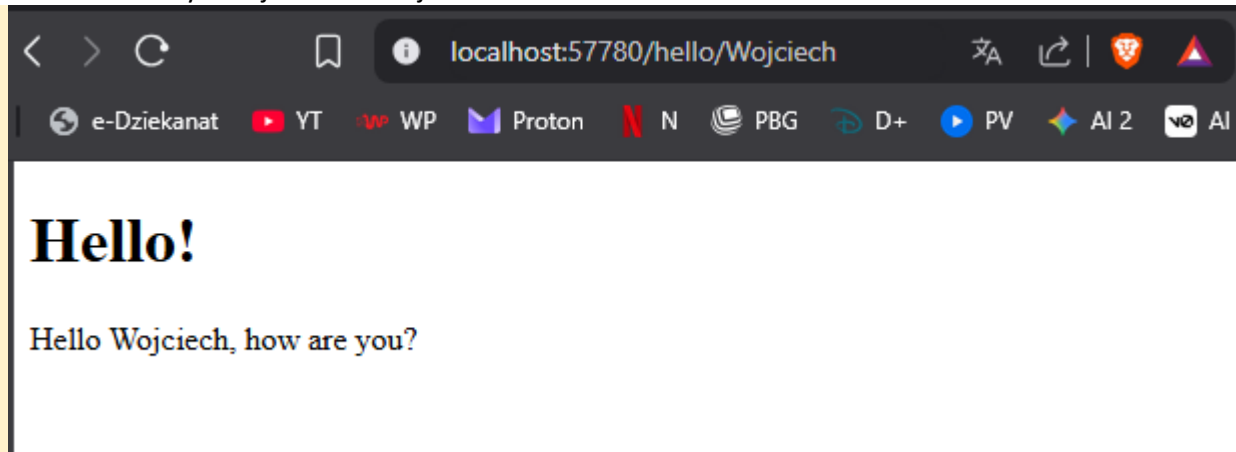
Wstaw zrzut ekranu kodu `hello.js` na tym etapie:

```
hello.js x hello.ejs
1 import express from "express";
2 const app : Express = express();
3 const port : number = 57780;
4 app.set("view engine", "ejs")
5 new *
6 app.get("/", (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<ResBody, LocalsObj> ) : void => {
7   res.send( body: "Hello World from Express!");
8 });
9 new *
10 app.get("/hello/:name", (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<ResBody, LocalsObj> ) : void => {
11   const name = req.params.name;
12   res.render( view: "hello", options: { name } );
13 })
14 new *
15 app.listen(port, hostname: () : void => {
16   console.log(`Hello World app listening on port ${port}`);
17 });
```

Wstaw zrzut ekranu kodu views/hello.ejs:

```
hello.js hello.ejs x
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Hello Express and EJS</title>
7 </head>
8 <body>
9   <h1>Hello!</h1>
10  <p>Hello <%= name %>, how are you?</p>
11 </body>
12 </html>
13
```

Wstaw zrzut ekranu strony <http://localhost:57780/hello/Wojciech> w przeglądarce:



W 3-5 zdaniach omów która część kodu odpowiada za to, że routing /hello/Wojciech zaczął działać, a imię zaczęło się wyświetlać w odpowiedzi:

`app.get("/hello/:name")` <- rejestruje dynamiczny route w expresie. `:name` to parameter url a express wyciąga jego wartość do `req.params.name`
`res.render(„hello”, {name})` <- zamiast `send` renderuje plik EJS i przekazuje do niego „name” a `<%=name%>` wyświetla imię

Punkty:	0	1
---------	---	---

PORTOWANIE APLIKACJI Z LAB G

W ostatniej części laboratorium przygotujemy kompletną aplikację CRUD (Create, Read, Update, Delete) na podstawie LAB G. Do rozpoczęcia nowego projektu wykorzystamy generator aplikacji Express. Przejdź terminalem do katalogu `C:\Users\akarczmarczyk\Desktop\ptw\lab-i` i wykonaj polecenia:

```
npx express-generator@4 --view=ejs --css=less crud
cd crud
pnpm install
```

Otwórz w PhpStorm projekt z katalogu `C:\Users\akarczmarczyk\Desktop\ptw\lab-i\crud`. Zapoznaj się ze strukturą plików i katalogów nowoutworzonego projektu.

W pliku `bin/www` zmodyfikuj port 3000 na swój własny (**57780**). Otwórz plik `bin/www` w edytorze i uruchom serwer w trybie zwykłym lub debug wybranym przyciskiem (1) lub (2) powyżej.

Odpowiedz na pytania:

Jaka wersja Express.js została wykorzystana?

4.16.4

Czy to najświeższa wersja?

nie

Jak zmienić domyślny port aplikacji z 3000 na 57780?

W pliku www

Gdzie znajdują się style less?

Public/stylesheets

Jak pogrupowane są routingi i ich handlers?

Index.js - /

Users.js - /users

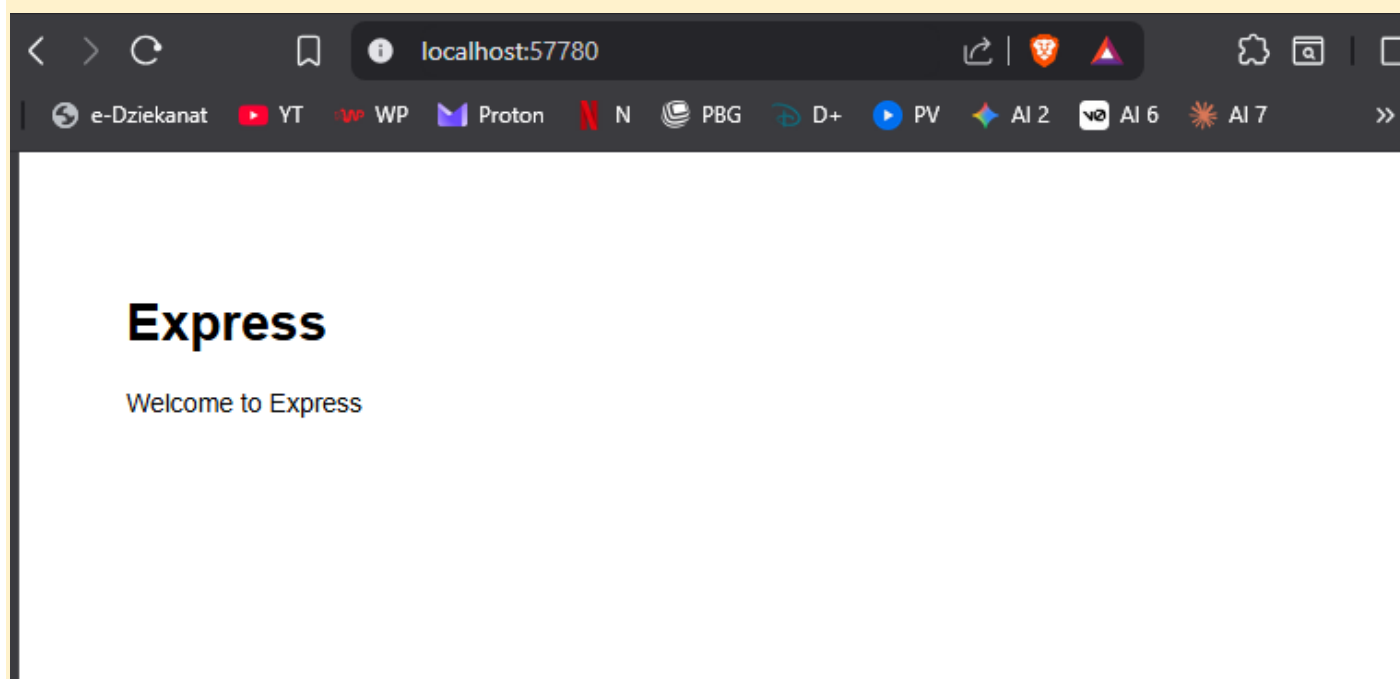
Gdzie znajdują się widoki EJS?

W folderze views

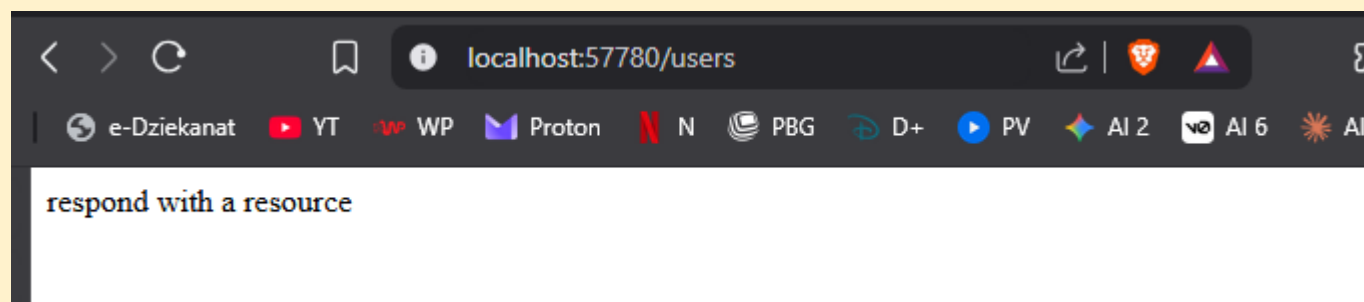
Dlaczego plik bin/www nie ma rozszerzenia .js?

Node nie wymaga tego

Wstaw zrzut ekranu strony <http://localhost:57780/> w przeglądarce:



Wstaw zrzut ekranu strony <http://localhost:57780/users> w przeglądarce:

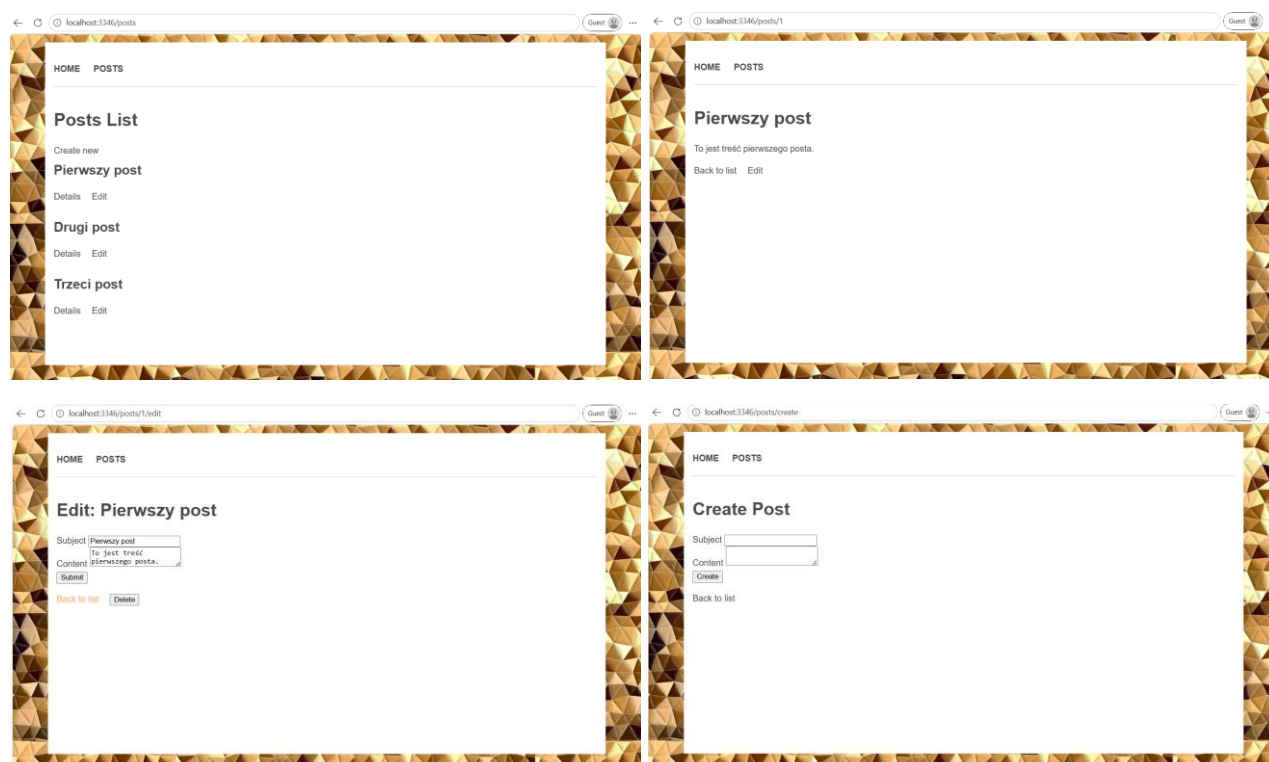


Punkty:	0	1
---------	---	---

IMPLEMENTACJA

W pozostałej części laboratorium, przenieś layout, style, migracje SQL (baza SQLite) oraz wszystkie operacje CRUD dotyczące Twojego modelu z aplikacji PHP do aplikacji w Express.js (tylko Twój model, bez postów).

Przykładowy efekt:



Hint! Obsługę SQLite w JS można osiągnąć na wiele różnych sposobów. Ja wybrałem taki:

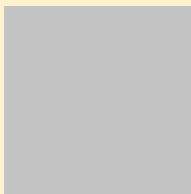
```

js posts.js x
1  var express : e | () => core.Express = require('express');
2  var router : Router = express.Router();
3  const { DatabaseSync } = require('node:sqlite');
4  const path = require('node:path');
5  const dbPath : string = path.resolve(__dirname, '..', 'data.db');
6  const db : DatabaseSync = new DatabaseSync(dbPath);

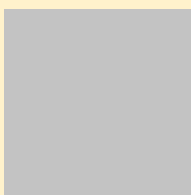
18 router.post( path: '/create', handlers: function(req : Request<P, ResBody, ReqBody, Re
19   try {
20     const { subject, content } = req.body;
21
22     const result : StatementResultingChanges = db
23       .prepare( sql: 'INSERT INTO post (subject, content) VALUES (?, ?)'
24       .run(subject, content);
25
26     res.redirect( url: '/posts/' + result.lastInsertRowid);
27   } catch (err) {
28     next(err);
29   }
30 });

```

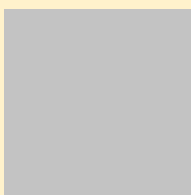
Wstaw zrzut ekranu kodu routingu i handlera listy:



Wstaw zrzut ekranu kodu szablonu EJS listy:

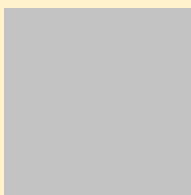


Wstaw zrzut ekranu listy (z danymi) w przeglądarce:

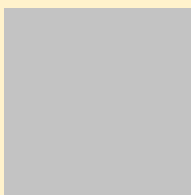


Punkty:	0	1
---------	---	---

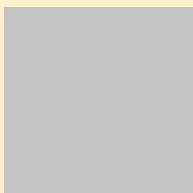
Wstaw zrzut ekranu kodu routingu i handlera podglądu pojedynczego modelu:



Wstaw zrzut ekranu kodu szablonu EJS podglądu:

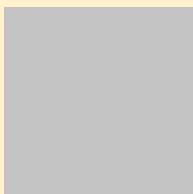


Wstaw zrzut ekranu podglądu w przeglądarce:

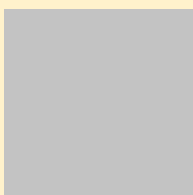


Punkty:	0	1
---------	---	---

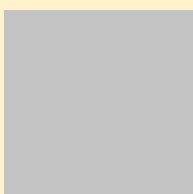
Wstaw zrzut ekranu kodu routingu i handlera tworzenia modelu:



Wstaw zrzut ekranu kodu szablonu EJS tworzenia:

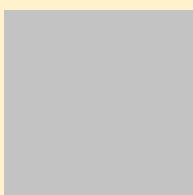


Wstaw zrzut ekranu tworzenia w przeglądarce:

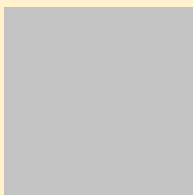


Punkty:	0	1
---------	---	---

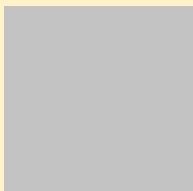
Wstaw zrzut ekranu kodu routingu i handlera(ów) edycji modelu:



Wstaw zrzut ekranu kodu szablonu EJS edycji:

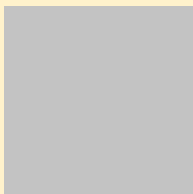


Wstaw zrzut ekranu edycji:

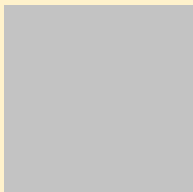


Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu routingu i handlera kasowania:



Wstaw zrzut ekranu działającego kasowania:



Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do swojego repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało.

Podaj link do swojego repozytorium:

...link, np. <https://github.com/.../...>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. **Nie używaj LLM.** Który z trzech poznanych dotychczas sposobów tworzenia aplikacji najbardziej przypadł Ci do gustu?

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.